

# Guía de Aprendizaje: Argumentos de línea de comandos en C

Malak Sanchez

Monitorías de Sistemas Operativos

## 1. Motivación

Muchos programas, especialmente en sistemas operativos Linux, no usan ventanas ni le piden al usuario que escriba texto interactivamente a lo largo de su ejecución. En su lugar, reciben las órdenes directamente al momento de ser ejecutados en la terminal, indicándoles qué archivo procesar o qué configuración usar. Para asimilar esos comandos iniciales, enviamos argumentos directamente a la función principal.

## 2. Idea Fundamental

La función `main` en C no tiene por qué estar vacía. Puede declararse para recibir el número de palabras que se escribieron en la terminal al momento de la ejecución, y un contenedor con cada una de esas palabras para ser analizadas dentro del programa.

## 3. Sintaxis Básica

```
int main(int argc, char **argv) {  
    // Cuerpo de la aplicacion  
    return 0;  
}
```

## 4. Ejemplo Mínimo Funcional

```
1 #include <stdio.h>  
2  
3 int main(int argc, char **argv) {  
4     printf("Total arguments: %d\n", argc);  
5  
6     for (int i = 0; i < argc; i++) {  
7         printf("Argument %d: %s\n", i, argv[i]);  
8     }  
9  
10    return 0;  
11 }
```

## 5. Explicación Paso a Paso

Si compilamos el código anterior en un ejecutable llamado **program** y lo ejecutamos en la terminal de la siguiente manera:

```
./program file.txt 42 hello
```

Dentro de nuestro programa, los valores se recibirán así:

- `argc` valdrá 4 (el nombre del programa y los 3 argumentos adicionales).
- `argv[0]` será `./program` (el comando con el que se invocó).
- `argv[1]` será `file.txt`.
- `argv[2]` será `42`.
- `argv[3]` será `hello`.
- `argv[4]` será `NULL` (por estándar, el vector termina de esta forma).

## 6. Qué ocurre en memoria

Cuando se corre un programa desde la consola (ej: `./program -a hello`), el sistema operativo agrupa los valores introducidos. El entorno de ejecución inicializa el programa reservando un arreglo de punteros específico, donde cada puntero dirige a una de las cadenas de caracteres correspondientes. Seguidamente, este bloque base de memoria es inyectado al inicio del **Stack Frame** de la función `main`.

## 7. Patrones comunes de uso

Validando formato y ejecutando conversiones numéricas de las cadenas de entrada:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main(int argc, char **argv) {
5     if (argc != 3) {
6         printf("Usage: %s <num1> <num2>\n", argv[0]);
7         return 1;
8     }
9
10    int val_a = atoi(argv[1]);
11    int val_b = atoi(argv[2]);
12
13    printf("Sum: %d\n", val_a + val_b);
14    return 0;
15 }
```

## 8. Errores Comunes

- **No chequear el tamaño de `argc`:** Tratar de acceder a valores como `argv[1]` sin comprobar primero que el usuario haya escrito suficientes elementos producirá un temido `Segmentation Fault`.
- **Tratar `argv` como valores matemáticos puramente:** Obviar que son cadenas de texto y pretender sumar los contenidos de `char *`, sin usar previamente conversiones al estilo `atoi` o `strtol`.

## 9. Buenas Prácticas

- Validar siempre `argc` de forma anticipada. Si los datos no están bien o no hay suficientes, terminar controladamente informando y educando al operador sobre cómo se debe invocar el archivo correctamente.
- Nombrar explícitamente a los parámetros como `argc` y `argv`; esto es una convención inquebrantable de la cultura en C.

## 10. Ejercicios

1. Escriba un programa que reciba su nombre como parámetro de ejecución y lo salude. Si no le pasan un nombre, indíquele el caso al usuario.
2. Codifique una calculadora básica de comandos que procese tres argumentos: un primer número, un signo para la instrucción (+, -, x), y un segundo número, retornado al sistema el resultado.